

Building Tide

Engine, UI & hardware

FOR ENGINEERS

How Tide is built and how to run it locally — the TypeScript engine, the Next.js demo UI, and the smart-plug hardware. Clone to running in about two minutes.

FOR

The dev team

UPDATED

May 22, 2026

STACK

Node 22 · Next 16

COVERS

code + hardware

Contents

The two codebases, the data flow, the build conventions, the gotchas, and the hardware.

- 1 TL;DR — running in two minutes
- 2 Prerequisites
- 3 The two codebases — and how they relate
- 4 Part A — the engine + CLI (prototypes/tide)
- 5 Part B — the demo UI (prototypes/tide-web)
- 6 Part C — the hardware (the plug)
- 7 How to extend
- 8 Testing & verification
- 9 Gotchas — read before you change anything
- 10 Pointers

For engineers on the team: how Tide is built, how to run it locally, and how to extend it — software and hardware both. The plain-English why is in [tide-team-brief.md](#); the end-user product flow is in [how-to-use-tide.md](#). This page is the build.

Status: these are two **prototypes**, not the official submission scaffold (that gets stood up at [/hackathon:scaffold](#) after the May 24 kickoff). The engine is proven; the packaging is deliberately throwaway. Read [prototypes/tide/VIABILITY.md](#) for the verdict.

TL;DR — running in two minutes

```
# 1. the engine + CLI (no browser, no hardware)
cd prototypes/tide
npm install
npm test           # 7 invariant tests on the optimizer + savings math
npm run sim        # full 24h simulation → decision → mock plug → savings, printed

# 2. the demo UI
cd ../tide-web
npm install
npm run dev        # http://localhost:3000 - the WAIT / GO NOW screen
```

That's the whole onboarding. No database, no env vars, no API keys — Tide reads a free public IESO feed and a checked-in fixture.

Prerequisites

Need	Why
Node 22+ (LTS)	Both projects use global <code>fetch</code> , <code>AbortSignal.timeout</code> , <code>node:test</code> , and ESM. 22 is the floor.
npm + git	Standard. No global tooling — <code>tsx</code> , <code>next</code> , <code>typescript</code> are all local devDeps.
a Shelly Plug US Gen4 (~\$25)	The physical relay — Tide is a hardware project . Optional only to run the <i>engine</i> locally (<code>MockPlugDriver</code> stands in); essential to the product and the demo. Full build in Part C .
(Kasa only) <code>tplink-smarthome-api</code>	If you use a Kasa plug instead of Shelly. Already in the spike's deps — but prefer Shelly (Part C explains why).

The *software* runs the same on Windows, macOS, Linux. The *hardware* is its own setup — [Part C](#).

The two codebases — and how they relate

```

prototypes/
├─ tide/           the ENGINE + CLI   (Node + tsx, TypeScript, ~520 LOC)
├─   └─ src/       pure decision logic + IO adapters + plug drivers
├─ tide-web/       the DEMO UI        (Next.js 16.2.6 + React 19)
├─   └─ lib/       a COPY of the engine's pure modules (see gotcha #1)

+ the HARDWARE     a Shelly smart plug on your LAN — the relay the engine
                   actually closes. MockPlugDriver stands in without it. (Part C)

```

The spike is the **source of truth** for the logic. `tide-web` carries a hand-copied subset of those modules so the Next app is self-contained. They share *concepts*, not *files* — see [Gotchas](#).

Part A — the engine + CLI (`prototypes/tide`)

Commands

Command	What it does
<code>npm run sim</code>	Full 24h simulation: prints the price/carbon curve, the optimizer's choice vs the naive baseline, the closed-loop plug transitions, and annualized savings. Start here.
<code>npm run live</code>	Probes the real IESO feed and parses the latest hour's fuel mix (needs internet). Proves the data dependency is real.
<code>npm run plug:test</code> <code>-- <ip></code>	Pure hardware handshake — ON → read-back → OFF → read-back. Run this first when a physical plug arrives.
<code>npm run plug --</code> <code><ip></code>	The real thing: read the grid, compute the window, switch a real plug for the current hour. Defaults to Shelly; append <code>kasa</code> for Kasa.
<code>npm test</code>	The 7 <code>node:test</code> invariants (no framework).
<code>npm run typecheck</code>	<code>tsc --noEmit</code> , strict. The CI-style gate.

Data flow

```

IESO fuel mix ┌
OEB rates    ┌─ HourSample[] → buildHorizon → chooseWindow → PlugDriver → ledger
ECCC factors └─ (ieso.ts)      (optimizer.ts)  argmin(cost    (plug*.ts)  (savings.ts)
                                         + λ·carbonKg)

```

File	Role
<code>src/types.ts</code>	All shared types — <code>FuelMix</code> , <code>HourSample</code> , <code>DayProfile</code> , <code>Load</code> , <code>WindowChoice</code> .
<code>src/rates.ts</code>	Ontario OEB rate schedules (ULO / TOU / TIERED) + <code>priceAt(hour, weekday, plan)</code> . ULO = 3.9¢ overnight / 39.1¢ on-peak.
<code>src/carbon.ts</code>	IPCC AR5 emission factors + <code>intensityFromMix(mix) → gCO₂/kWh</code> . (Average, not marginal — see gotcha #3.)
<code>src/ieso.ts</code>	<code>fetchLatestFuelMix()</code> (live, never throws — returns a structured <code>LiveResult</code>) + <code>loadSampleDay()</code> (the fixture, run through the same pipeline).
<code>src/optimizer.ts</code>	The brain. <code>buildHorizon</code> → <code>chooseWindow</code> (argmin over feasible start times of <code>cost + λ · carbonKg</code>) + <code>baselineNow</code> . Pure, no I/O, fully unit-tested.
<code>src/savings.ts</code>	<code>perRun(chosen, baseline)</code> and <code>annualScenarios()</code> .
<code>src/plug.ts</code>	The <code>PlugDriver</code> interface (<code>name / on / off / isOn</code>) + <code>MockPlugDriver</code> .
<code>src/plug-shelly.ts</code>	Recommended real driver — Shelly Gen2+ local JSON-RPC, zero deps, no cloud.
<code>src/plug-kasa.ts</code>	TP-Link Kasa driver (optional dep; KLAP/cloud caveat in the header — prefer Shelly).
<code>data/sample-day.json</code>	Deterministic Ontario-summer fixture (<code>{date, weekday, hours:[{hour, mix}]}</code>).
<code>web/index.html</code>	A dependency-free static version of the screen (open it directly).

Conventions (the spike will not compile if you break these)

- **ESM + explicit `.ts` import extensions.** `tsconfig` is `NodeNext` + `verbatimModuleSyntax` + `allowImportingTsExtensions` , so imports read `from "./optimizer.ts"` . Keep the extension or `tsc` errors.
- **`tsx` runs, `tsc` only checks.** There is no build step — `tsx` executes TypeScript directly (`noEmit: true`). `npm run typecheck` is purely a gate.
- **Pure core, IO shell.** `optimizer.ts` , `carbon.ts` , `rates.ts` , `savings.ts` are pure and tested. Network and hardware live only in `ieso.ts` and the `plug-*.ts` drivers. Keep that line — it's why the brain is testable without a grid or a plug.
- **The live fetch never throws.** `fetchLatestFuelMix` always resolves to a `LiveResult` with an `ok` flag. The demo must survive a dead feed; the data is a bonus, not a crutch.

Part B — the demo UI (`prototypes/tide-web`)

Next.js 16.2.6 App Router + React 19. No Tailwind, no UI kit — one screen, hand-written CSS in `app/globals.css`.

Commands

Command	What it does
<code>npm run dev</code>	Dev server at <code>http://localhost:3000</code> .
<code>npm run build</code>	Production build (needs no network — the live reading is render-time only).
<code>npm start</code>	Serve the production build.

How it's wired

```

app/page.tsx      SERVER component, `export const dynamic = "force-dynamic"`
├─ getDay()        fixture → intensity + prices (lib/grid.ts)
├─ buildHorizon → chooseWindow → baselineNow → perRun  (the decision, server-side)
├─ await getLiveIntensity()  real IESO read, cached 300s, returns null on failure
└─ <TideScreen ... />  passes plain props to the client

app/tide-screen.tsx  CLIENT component ("use client")
└─ the WAIT / 60 verb + 24h price·carbon strip + hour scrubber (useState) + savings tally
  
```

- `@/lib/*` **path alias** (Next default) — `lib/` holds the copied engine: `optimizer.ts`, `rates.ts`, `carbon.ts`, `savings.ts`, `types.ts`, plus `grid.ts` (the spike's `ieso.ts`, renamed; uses Next's `fetch(url, { next: { revalidate: 300 } })`) and `sample-day.ts` (the fixture as a TS module, not a JSON read).
- **No plug drivers here.** `tide-web` is display-only. Actuation lives in the spike — the eventual real app wires them together.
- **Palette is data-anchored, not decorative:** teal = clean/cheap ($\leq 4\text{¢}$), blue = mid, orange (`#c84b1f`) = on-peak ($\geq 39\text{¢}$). Defined in `tide-screen.tsx`'s `colorFor`.

Part C — the hardware (the plug)

Tide is not a dashboard — it's a relay that physically closes on the cheap hour. The engine runs fine on a `MockPlugDriver`, but the product and the on-stage demo need a real plug. This is the part that has lead time and the part most likely to fail live, so treat it as a first-class deliverable, not an accessory.

Bill of materials

Item	~Cost	Why
Shelly Plug US Gen4 (or Plus Plug US, ~\$20)	~\$25	The relay. Gen2+ Shelly speaks a documented local JSON-RPC HTTP API with no cloud account — the one plug that reliably survives hostile venue Wi-Fi.
A 120 V load to switch	—	EV Level-1 charger, window AC, dehumidifier, pool pump, dishwasher. For the demo: a desk lamp standing in for "the dishwasher" — visible, instant, safe.
Travel router (GL.iNet) or a phone hotspot	~\$30	Own the LAN at the venue. Do not trust conference Wi-Fi (client isolation + captive portals break a shared-LAN demo).
<i>(fallback only)</i> older Kasa KP115/HS103/EP10	~\$15	A flashed, known-good backup if the Shelly fails.

Order the Shelly now — it has to arrive before the in-person finale (May 29–30).

⚠️ 120 V plug-through loads only. The plug switches a normal wall outlet. Never put a 240 V dryer, Level-2 charger, central AC, or hardwired water heater on it, and don't exceed the plug's amp rating. This is the same limit the [product guide](#) states for users — it's a hard electrical boundary, not a software one.

Bring up the Shelly (~10 min, once)

1. Power it; join its setup Wi-Fi; point it at your network (**2.4 GHz** — these plugs don't do 5 GHz).
2. **Turn Cloud OFF** in the web UI. This is the whole trick: control now stays on your LAN, auth is off by default, and you dodge the cloud-firmware lottery entirely.
3. Find its IP and **reserve it** (static, by MAC address) in your router so it never moves. Hard-code that IP wherever you pass it to Tide.
4. Confirm the API straight from a browser or `curl` before involving any code: `http://<plug-ip>/rpc/Switch.Set?id=0&on=true # relay ON http://<plug-ip>/rpc/Switch.GetStatus?id=0 # → { "output": true } http://<plug-ip>/rpc/Switch.Set?id=0&on=false # relay OFF` That is the *entire* protocol `src/plugin-shelly.ts` wraps — plain HTTP, zero dependencies, 2-second timeout so a flaky LAN can't hang the demo.

Test it from Tide

```
cd prototypes/tide
npm run plug:test -- <plug-ip>      # pure handshake: ON → read-back → hold 2s → OFF → read-back
npm run plug -- <plug-ip>           # then the grid-aware one-shot (Shelly default; append `kasa` for Kasa)
```

If the lamp blinks on, holds, then off — the LAN control path is sound and you can trust it on stage. `plug:test` deliberately has **no grid logic**, so it isolates "can my laptop switch this plug?" from "did the optimizer pick the right hour?"

Demo-day network checklist

- Bring the travel router; pre-configure its SSID + WPA2 **at home**, not in the room.
- Join the laptop **and** the plug to *only* that router; disable the laptop's auto-join to other networks.
- No internet uplink is required — local RPC works on an island network with no WAN.
- Toggle the plug once on-site during setup, before the room fills, to confirm the reserved IP took.

Kasa fallback (avoid if you can)

`src/plugin-kasa.ts` exists and works, but newer TP-Link **KLAP** firmware demands your TP-Link **cloud** credentials even for "local" control, and KLAP-v2 has disabled the local port on some models entirely. If you're forced onto Kasa: use an **older single plug (KP115 / HS103 / EP10)**, **factory-reset**, and **never connect it to the Kasa app/cloud** so it stays on the credential-free legacy protocol — then `npm i tplink-smarthome-api`. Shelly removes this whole failure class; that's why it's the default.

If the radio dies entirely

The closed loop is *software + a relay* — the relay is the upside, not a single point of failure. With no hardware reachable, `MockPlugDriver` (and the on-screen WAIT / GO state) still demonstrate the decision and the state change. Always have the screen-only path rehearsed as the fallback, so a dead plug costs you the *flourish*, not the *demo*.

How to extend

- **New region (Québec, NYISO, UK Octopus...):** swap exactly two things — the rate schedule in `rates.ts` and the IESO adapter (`ieso.ts` / `grid.ts`). The optimizer, drivers, savings, and UI are region-agnostic. The product is the scaffold; the region is config.
- **New plug brand:** implement the 4-method `PlugDriver` interface. Copy `plug-shelly.ts` as the template — `connect()` should probe once so an unreachable plug fails fast.
- **Greener vs cheaper:** the `lambda` arg to `chooseWindow` is a shadow carbon price in \$/kg. `0` = pure cost (the demo default); raise it to weight carbon. Invariant test: raising λ never *increases* carbon.

Testing & verification

```
cd prototypes/tide && npm test && npm run typecheck # engine: 7 tests + strict types
cd ../tide-web && npm run build # UI: production build is green
```

The engine tests (`test/optimizer.test.ts` , `savings.test.ts`) assert **behavioural invariants**, not snapshots: the optimizer picks an overnight window, the smart window beats charging-on-plug-in, raising λ never adds carbon, an infeasible load throws. Keep new logic invariant-tested the same way.

Gotchas — read before you change anything

1. **The engine is duplicated.** `prototypes/tide/src/{optimizer,rates,carbon,savings,types}.ts` and `prototypes/tide-web/lib/*` are **parallel copies**. Change the optimizer in one and the other silently drifts. This is a deliberate hackathon shortcut; the real scaffold should make it one shared module. Until then: **change both**.
2. **The XML parser is a regex.** `ieso.ts` / `grid.ts` grab the last `<HourlyData>` block with a regex. It works on the real feed today but is brittle to schema drift — production swaps in `fast-xml-parser`.
3. **Carbon factors are average, not marginal.** Cost savings are *exact*; carbon is *directional*. The honest framing (cost-first, carbon-qualified) is in `carbon.ts` 's header — don't overclaim net-carbon reduction.
4. **The load is hard-coded.** `page.tsx` 's `LOAD` and the scripts' `Load` objects (1.44 kW EV charger, 6h, plugged in 18:00, due 31:00 = 7am) are not user-configurable yet. That's the "tell Tide your load" feature the product still needs.
5. `force-dynamic` means every request hits IESO (cached 300s via `revalidate`). The build itself needs no network — keep it that way so deploys never depend on the grid being up.
6. **Hardware needs a real LAN, not venue Wi-Fi.** Turn the Shelly's **Cloud off**, reserve a static IP, run it on your own travel router — full setup and the no-radio fallback are in [Part C](#). `MockPlugDriver` is the screen-only fallback.
7. **These are prototypes, not the scaffold.** Don't build the submission directly on top of either without the `/hackathon:scaffold` consolidation decision — that's gated to post-May-24 idea-lock.

Pointers

- `tide-team-brief.md` — why Tide exists, in plain English (share with non-engineers)
- `how-to-use-tide.md` — the end-user product flow
- `prototypes/tide/README.md` — the spike's own run notes
- `prototypes/tide/VIABILITY.md` — what's proven, what's still open